

KNOWLEDGE MAP · LEARNING HANDBOOK

AI Agent 开发知识体系 学习手册

从认知地基到规模化工程 · 一本学完，不再东拼西凑

配套文件：《AI-Agent知识体系-思维导图.html》（可交互导图，可折叠缩放）

整理自「AI Agent」知识库 13 篇专栏文章 · 2026 年 9 月

目录与学习脉络

本手册按「**认知** → **设计模式** → **工具协议** → **可靠性** → **记忆** → **评估** → **规模化**」的顺序编排，每一章都是下一章的地基，按顺序读就是一条完整的学习路线。

1. **第 1 章 • Agent 认知** —— Agent 到底是什么，与普通问答 / ChatBot / Workflow 的本质区别
2. **第 2 章 • 设计模式与选型** —— ReAct / Reflection / 规划执行，以及什么时候根本不需要 Agent
3. **第 3 章 • 工具与协议** —— 工具设计决定 Agent 上限；MCP 协议详解
4. **第 4 章 • 可靠性兜底** —— 死循环、误调用、上下文污染、权限越界怎么兜
5. **第 5 章 • 记忆与状态** —— 短期记忆 / Session State / 长期记忆 / RAG 的关系
6. **第 6 章 • 评估与度量** —— 任务完成率、步骤效率、错误恢复率与可靠性度量
7. **第 7 章 • 规模化工程** —— DAG 执行器、Planner/Worker/Reviewer 分工、上下文与 Token 治理、Browser Agent 读大网页
8. **附录 A • 知识框架总览、重点难点与复习建议**
9. **附录 B • 思维导图（树形文本版）**

学习主线：先认知 → 再设计模式 → 再工具 → 再兜底 → 再记忆评估 → 最后规模化工程

学习目标：不是“搭一个能跑的 Demo”，而是搞懂 Agent 为什么转得动、为什么翻车、翻车了怎么兜底。若 Prompt 与 Function Calling 的底子已打好，Agent 这一段大约两到三周能走完一遍，再配一个会查资料、会调工具、能自己规划步骤的小项目即可。

第 1 章 • Agent 认知

本章主线：普通问答只会"生成答案"，Agent 会"围绕目标持续行动"。抓住四个能力（规划、工具、执行、反馈）和一个判断标准（有没有自己决定下一步），就能把 Agent 从热词讲成工程系统。

1.1 普通大模型问答：本质是"你问我答"

普通问答的交互是：**用户给问题** → **模型生成答案** → **用户继续追问**。它可以讲得很细、能写代码、能总结文章，但核心特点是——**它主要是在生成文本**。它不知道数据库里有什么数据，不会自己去查监控、打开后台、执行脚本，也不会在第一步失败后自动换方案。普通问答像一个知识很强的顾问：能给建议，但行动还得你来做。

1.2 Agent 的关键：不是"更会说"，而是"更会做"

普通问答的核心产物是"答案"；Agent 的核心产物是"完成任务的过程和结果"【对比】。

以"分析网站转化率为什么下降"为例：

- **普通问答：**给一套分析思路（看流量来源、看加载速度、看点击率.....），数据还得你自己查。
- **真正的 Agent：**①拆解问题（流量/页面/价格/性能/用户路径）→ ②调数据工具查 30 天 PV、UV、点击率、支付转化 → ③调日志工具看延迟、错误率 → ④对比历史找异常时间点 → ⑤生成初步结论 → ⑥继续验证（是否与某次发布有关）→ ⑦输出结论、证据与修复建议。

一句话记忆
Agent 不是"回答问题"，而是围绕一个目标，自己决定下一步做什么，并不断根据结果调整。

1.3 Agent 至少要有四个能力

能力	内涵	易错 / 补充
规划	把目标拆成任务：先看什么、再看什么、最后输出什么	没有规划就会"想到哪做到哪"；很多 Agent 翻车就是计划不稳定——一开始说要查测试，跑着跑着忘了
工具	连接外部世界：搜索、数据库、文件读写、代码执行、浏览器、业务 API、发消息接口	工具让模型从"文本生成器"变成"能操作环境的执行者"；Function Calling / Tool Use 是行动能力的入口，不是附属功能
执行	一步一步把任务跑完，能在执行中处理分支	会列计划 ≠ 会执行；查库失败、返回为空、接口超时、数据不够时要不要换工具，都属于执行
反馈	看结果，再决定下一步：思考 → 行动 → 观察结果 → 再思考 → 再行动	发现失败最多的是"支付超时"，下一步就不该继续查库存，而应去查支付接口延迟、回调、网关日志（这就是 ReAct 的思路）

1.4 一个简单判断：它有没有"自己决定下一步"

- 固定流程「输入 → 检索知识库 → 模型总结 → 返回」更像 **RAG 问答**，不一定是 Agent。
- 「给目标 → 判断先查知识库 → 信息不够又查数据库 → 数据异常又调日志工具 → 根据日志继续定位 → 生成结论」——根据任务状态做**动态决策**，这才更像 Agent。
- 不是用了大模型就叫 Agent，也不是用了 Function Calling 就叫 Agent：如果工具调用路径完全固定，它只是**带工具的 Workflow**。

核心公式（面试可直接说）

Agent = 目标驱动 + 动态决策 + 外部行动 + 反馈迭代

1.5 Agent 不是"一个更强的模型"，而是一套工程系统

一个完整 Agent 系统至少包括九个组件：

- **模型**：理解、推理、生成；**Prompt**：定义角色、目标、边界；**工具**：连接外部系统；**执行器**：真正调用工具、处理返回结果
- **状态管理**：记录当前任务进度；**记忆系统**：保存跨轮信息；**评估机制**：判断做得对不对
- **权限控制**：限制能做什么；**失败恢复**：出错后能否重试、回滚、停止

易错点

同样用一个模型，不同团队做出来的 Agent 效果差很多——差距往往不在模型本身，而在**工具设计、上下文管理、执行编排、评估观测**这些工程细节。模型负责思考，系统负责让它安全地行动。

1.6 学习地基：结构化输出 + Function Calling

- **结构化输出**：Agent 的每一步决策都依赖模型吐出可解析的结构（如 JSON Schema 约束）；输出不稳定，Agent 直接散架【因果】。
- **Function Calling**：Agent 之所以能"动手"，靠的就是它去调外部工具——这是 Agent 的命根子，必须先吃透。

面试怎么回答"Agent 是什么"

"我理解的 Agent，不是单纯的大模型问答，而是一个围绕目标持续行动的系统。它会根据用户目标进行任务规划，选择合适的工具调用外部环境，执行中观察工具返回结果，再决定下一步动作。普通 ChatBot 主要生成回答，而 Agent 强调行动、反馈和迭代。它的核心不是模型更聪明，而是系统具备规划、工具调用、执行编排和状态反馈能力。" 被追问与 Workflow 的区别时可补："流程能写死，用 Workflow 往往更稳定、更可控"（详见第 2 章）。

第 2 章 · 设计模式与选型

本章主线： ReAct、Reflection、规划执行不是三套神秘框架，而是三种朴素的做事方式——它们都在回答同一个问题：Agent 应该怎么推进任务。同时，**不是所有任务都需要 Agent**，会讲“什么时候不用 Agent”比会写十个 Demo 更值钱。

2.1 底层共识：Agent 本质是一个循环

看到目标 → 想下一步 → 调工具 → 看结果 → 再想下一步。不是一次性给答案，而是每一步都根据上一步结果继续判断。三种模式的区别只在“推进方式”不同【并列】。

2.2 ReAct：边思考，边行动，边观察

- **Reasoning**（推理：先想清楚下一步为什么这么做）+ **Acting**（行动：调用工具或执行动作），中间靠 **Observation**（工具执行后的观察结果）把结果带回来。
- 典型循环：Thought（我需要知道什么）→ Action（调哪个工具）→ Observation（返回了什么）→ Thought（下一步做什么）。
- 例：定位 `/api/pay` 变慢——查监控发现 22:10 后 P95 从 300ms 涨到 3.8s → 查发布记录发现 22:05 发布了 v2.3.1 → 查日志发现第三方支付回调超时激增。
- 特点：不要求一开始想完完整计划，每拿到一个结果再决定下一步——所以特别适合**路径不确定**的工具调用场景。

为什么面试官最爱问 ReAct

- 确认你是否理解**工具调用不是一次性的**：调用什么工具要看上一步结果。
- 确认你是否理解 **Observation 的重要性**：没有 Observation，模型就是在脑补；返回是否为空、是否异常都会影响下一步。
- 确认你是否知道 **ReAct 也会翻车**：每步都动态决策，没有约束就容易忘掉原始目标、一直调无关工具、在失败结果里反复重试、信息够了还继续查。

加分句

"ReAct 适合路径不确定的任务，但必须配合**步数限制、工具权限、状态记录和停止条件**。"

2.3 Reflection：交付前按标准检查

- Reflection 的核心不是一句提示词（“我再仔细检查一下”没用），而是给 Agent 一个**明确的检查环节**：写完 SQL 检查字段、WHERE 条件、是否全表扫描、结果是否符合目标；写完报告检查结论有无证据、有无遗漏关键时间点、是否把相关性当因果。
- **三类适用场景**：①结果容易有格式错误（结构化输出、SQL、配置、接口参数）；②任务有明确验收标准（单测通过、JSON 符合 Schema、报告包含指定字段）；③Agent 容易过早交付（交付前加一道门：目标是否满足？还缺什么证据？）。

- **边界**：如果模型本身不知道正确答案长什么样（没有日志、监控、发布记录只靠自我反思），反思十遍也没用——最好和工具、规则、测试、评估一起用，不要当玄学许愿。

2.4 规划执行 (Plan-and-Execute)：先拆任务，再推进

- **逻辑**：先让模型根据用户目标生成计划，再按计划一步一步执行。以"分析仓库技术债"为例：了解项目结构 → 找核心模块和高频变更文件 → 检查复杂函数、重复代码、缺测试模块 → 结合影响范围评优先级 → 输出清单与建议。
- **优点**：任务更有方向，不容易一开始就跑偏；适合复杂任务（代码库分析、竞品调研、数据分析报告、多文件重构、长文档整理）。
- **最大问题**：计划经常跟不上变化——一开始以为问题在数据库，查完发现没问题，再查才发现是缓存击穿；死守计划会越做越偏。

工程化答法

"我们不会让 Agent 生成一个计划后机械执行到底，而是把计划当作**可更新的任务清单**。每完成一步都记录状态和证据，如果发现假设不成立，就重新规划后续步骤。"（落地细节见第 7 章 DAG 执行器）

2.5 三种思路怎么选

场景	更适合的思路	原因
路径不确定，需要边查边判断	ReAct	每一步根据工具结果决定下一步
结果容易出错，需要交付前检查	Reflection	用标准拦截格式、逻辑和证据问题
任务复杂、步骤多、容易跑偏	规划执行	先拆任务，避免一开始就乱跑
代码修改、数据分析、故障排查	组合使用	先规划，中间 ReAct，交付前 Reflection

例："定位线上支付接口变慢并给出修复建议"的稳态设计：**先规划**（确认要查监控、日志、发布记录、下游依赖）→ **中间 ReAct**（根据每次查询结果决定下一步查哪里）→ **最后 Reflection**（检查结论是否有证据、建议是否对应根因）。

三句话总结

ReAct 解决"下一步怎么根据结果继续走"；Reflection 解决"交付前怎么发现自己做错了"；规划执行解决"复杂任务怎么先拆开再推进"。真正的 Agent 不是堆模式名，而是知道任务需要什么能力，再组合这些思路。

2.6 选型边界：Agent vs Workflow

两条基本原则

- **流程能写死，就优先 Workflow**。Workflow 是步骤固定、规则明确、输入输出稳定的流程编排（如发票审核：OCR → 提取字段 → 校验 → 入账 → 返回），核心价值是把业务流程变成一条可控链路：每步做什么、依赖什么输入、失败怎么重试、哪里要人工审批、结果怎么记录追踪。

- **路径不确定、需要中途判断、需要多工具探索，才考虑 Agent。**如"定位线上接口变慢"：可能是数据库慢、缓存击穿、发布引入慢查询、第三方超时——没法提前写死每一步。

易混点: "有分支" ≠ "需要 Agent"

关键看**分支能不能提前定义**。退款流程（未发货自动退 / 已发货人工审 / 超期拒绝）、风控审核（金额阈值 / 黑名单）都是**规则分支**，条件清楚、边界明确，适合 Workflow——让 Agent 判断反而可能解释错规则、在边界条件上犯错。真正需要 Agent 的是**开放决策**：没法提前列出完整路径、查到一个异常后下一步要跟着异常走。

过度 Agent 化的四个代价

代价	说明
成本更高	Workflow 几次接口调用；Agent 要多轮模型调用、多次工具调用、反复观察总结，还消耗大量 token
延迟更长	查个订单状态直接查库即可；让 Agent 理解意图、规划、调工具、总结只会让体验变差
稳定性更差	动态决策意味着灵活也意味着不稳定：可能选错工具、重复调用、提前总结、跑偏
调试更难	Workflow 看节点日志一眼定位；Agent 要看整条 trace，为什么选这个工具、为什么忽略那个结果

实用判断法与混合架构

- **先问：这件事能不能画成固定流程图？**能画出来且覆盖大多数情况 → Workflow（发票审核、订单退款、内容审核、工单流转、数据同步）。画不出来或分支爆炸 → Agent（故障定位、竞品调研、大型代码库分析、反馈归因、多数据源经营分析）。
- **最稳的方案是 Workflow + Agent：**外层 Workflow 控制主流程（接收工单、判类型、分配队列、通知、记录），"分析工单原因"这类开放节点交给 Agent。**Workflow 负责边界和流程，Agent 负责开放问题求解。**

上 Agent 前的四问

① 这个流程能不能写死？② 分支规则能不能提前定义？③ 错了以后影响大不大？④ 用户更需要灵活，还是更需要稳定？答案偏向稳定，就别硬上 Agent。

面试怎么回答

"Workflow 适合步骤明确、规则稳定、可提前编排的任务，核心是稳定执行；Agent 适合目标明确但路径不确定、需要中途观察结果并动态决策的任务，核心是探索和调整。工程上如果流程能写死，用 Workflow 更稳定、更便宜、更容易审计。我们一般把主流程放在 Workflow 里保证权限、状态流转和异常处理可控，只有在路径不确定的节点（故障定位、原因分析、复杂数据查询）才让 Agent 调工具动态判断。"

第 3 章 · 工具与协议

本章主线：很多 Agent 不是模型不行，而是工具设计太差。工具是 Agent 连接外部世界的接口，也是约束 Agent 行为的边界——工具设计好了，Agent 才有上限；MCP 则解决“工具怎么标准化接入”的问题。

3.1 工具不是 API 列表，而是 Agent 的行动边界

后端 API 是给程序员用的（知道业务含义、调用时机、参数来源、异常处理）；Agent 工具是给模型决策用的（模型只看到工具名、描述、参数 Schema 和历史上下文）。把 `queryData / getInfo / submit / updateStatus` 这类接口原样丢给模型，它只能猜。

第一条原则

工具不是把 API 暴露给模型，而是把可控动作封装成**模型能理解、能选择、能恢复**的能力。一个好工具至少回答四问：能做什么？什么时候该用？什么时候不该用？调用完后 Agent 能否根据结果继续判断？

3.2 工具描述：不要只写“查询订单信息”

Agent 要靠描述判断“下一步该不该用它”。差的描述：“**查询订单信息**”；好的描述：写明查询哪些状态、适用于什么问题、**不用于**创建/修改/退款。描述要写清三类信息：

- **能解决什么问题：**如“查询订单状态”“查询接口延迟”，不写“处理用户请求”这类空话；
- **不能做什么：**查询工具不能修改数据、草稿工具不能直接发送——Agent 最怕把相似工具混在一起；
- **什么时候优先使用：**如“用户提供订单 ID 时优先使用”“知识库检索无结果时不要反复调用，应改为追问用户”。

工具描述不是注释，工具描述就是 Agent 的使用说明书。

3.3 参数设计：别把脏活都丢给模型

粗暴的 `{"query": "string"}` 让模型自己决定查哪个服务、什么时间段、什么级别——它可能把时间写成“昨天晚上”、把范围查太宽导致超时。应拆成 `service_name / start_time / end_time / level(enum) / keyword` 等结构化参数。四条实用原则：

1. **能枚举就枚举：**日志级别、订单状态、排序方式用 enum，避免 `err / error / ERROR_LOG / fatal` 式乱填；
2. **时间、数量、范围要明确：**要求 `start_time / end_time / limit / page_size` 格式，并加上限（日志最多 100 条、检索最多 5 个文档块、监控范围不超过 7 天）——防止一次调用拖垮系统；
3. **一个参数不承担多个含义：**`{"user_input": "查张三昨天的订单"}` 混了用户、时间、对象和意图，应拆成 `user_name / date / query_type`；
4. **缺关键参数时不要硬编：**用户没给订单号，Agent 应追问“请提供订单号”，而不是编一个——很多翻车就是在关键信息缺失时硬调用，那不是解决问题，是制造幻觉。

3.4 返回值设计：别只返回一段自然语言

返回 "订单已支付，正在仓库处理中，预计明天发货"，人能看懂，但 Agent 无法稳定判断要不要继续查物流、怎么告诉用户。结构化返回至少包含三类信息：

类别	字段示例	作用
状态字段	success / status / error_code / order_status / risk_level	让 Agent 做分支判断
证据字段	数据来源、命中文档、查询时间范围、监控指标、日志片段 ID	让最终回答"根据什么得出结论"，不只是猜
下一步提示	suggested_next_actions：如问物流可调 get_shipping_detail	给 Agent 可靠的行动提示，比凭空猜稳定

3.5 工具粒度：太细会跑断，太粗会失控

- **太细** (get_user_id_by_phone → get_order_ids_by_user_id → ...)：一个简单问题要连调五六次，中间任何一步断了就全断——Agent 变成脆弱的流程编排器；
- **太粗** (handle_order_problem / solve_user_issue)：内部做了什么 Agent 不知道，返回"处理成功"也不知道成功在哪——系统变黑盒，不可解释、不好调试；
- **合理粒度**：一个工具完成一个清晰的**业务动作**并返回可判断的结构化结果，如 get_order_detail、check_refund_policy、create_refund_draft、search_service_logs、query_api_latency、get_deploy_records——介于底层 API 和完整业务流程之间。

3.6 有副作用的工具，必须单独设计边界

工具分两类：**查询类**（只读，错了最多答案不准）和**动作类**（改变外部世界，错了可能造成业务事故）。两类绝不能混在一起设计。动作类工具至少三层保护：

1. **读写分开**：不要设计模糊的 handle_refund；应拆成 get_order_detail / check_refund_policy / create_refund_draft / submit_refund_after_confirm——读是读、写是写、草稿是草稿、正式提交是正式提交；
2. **高风险动作必须二次确认**：凡影响钱、权限、数据、通知、生产环境（退款、扣费、删数据、改权限、发邮件、重启服务、改生产配置），都不要让 Agent 单独决定——先生成草稿或计划，用户确认后再执行；成熟的 Agent 系统必须知道什么时候停下来让人确认；
3. **返回要记录审计信息**：操作人、对象、时间、前后状态、审计 ID、是否需要人工复核。这样 Agent 会说"我已创建退款申请草稿，需要你确认后才会正式提交"，而不是"我已经退款了"。

3.7 错误信息要能让 Agent 恢复

只返回 "系统异常" 对 Agent 没帮助——它不知道该重试、换参数、换工具还是告诉用户。好的错误返回应结构化：`error_code + recoverable + recommended_action`（+ retry_after / missing_fields /

permission_required)。例如缺参数 → ask_user_for_order_id；时间范围过大 → narrow_time_range；权限不足 → stop_and_explain。

重要洞察

很多幻觉不是从模型生成开始的，而是从工具返回值太差开始的。如果工具失败后 Agent 无法恢复，它就很容易乱编一个答案。

3.8 不要一次性把所有工具都塞给 Agent

"工具越多越强"不一定成立：选择空间越大，选错概率越高——这对模型不是增强，是干扰。应按任务动态收敛工具集：客服订单问题只给订单/物流/退款规则/退款草稿/追问工具；排查故障只给监控/日志/发布记录/链路追踪/报告工具；写代码只给文件读取/代码搜索/测试执行/补丁编辑。外层 Workflow 可以先判断任务类型，再给 Agent 分配对应工具箱。

3.9 工具设计不好，Agent 会出现哪些症状

症状	根因	对策
经常选错工具	工具描述太像、边界不清	写清适用与不适用场景
参数经常填错	参数 Schema 太松	收紧类型、枚举、必填项和格式说明
调用后不知道下一步	返回值没有状态与证据字段	返回结构化结果
重复调用同一工具	缺少失败语义和停止条件	标记无结果原因，提示追问用户或换策略
做了不该做的动作	读写边界不清、无二次确认	拆分查询、草稿和正式执行工具

3.10 工具设计 Checklist (14 条)

工具名能否一眼看出业务动作 | 描述是否写清适用场景 | 是否写清不适用场景 | 必填参数是否明确 | 能枚举的是否用 enum | 时间/数量/范围是否有格式和上限 | 缺参数时是追问还是硬编 | 返回值有无状态字段 | 有无证据字段 | 错误信息能否指导恢复 | 查询与写操作是否分开 | 高风险动作有无二次确认 | 调用有无审计 ID | 当前任务是否真需要这么多工具

3.11 MCP 协议：Agent 工具调用的新标准

为什么会有 MCP

没有统一协议时，接 GitHub、数据库、本地文件、Slack、Sentry 各写一套；给 Claude 接一遍、给 Cursor 接一遍、给内部 Agent 再接一遍——越做越像胶水工程。MCP (Model Context Protocol) 是一套标准协议，让 AI 应用以统一方式连接外部工具、数据和提示模板。它不是模型，也不是 Agent 框架，更像 Agent 时代的"工具接入协议"——就像前后端之间用 HTTP API 通信。

MCP 不是 Function Calling 的替代品

	Function Calling	MCP
解决什么	模型这一次响应里要不要调用某个函数、参数是什么	工具从哪来、怎么发现、怎么连接、怎么暴露给不同 AI 应用
所在层面	模型侧的调用机制	应用侧的接入协议
关系	互补不互斥：很多应用先通过 MCP 发现工具，再转成 Function Calling / Tool Use 格式让模型决定调用	

核心架构：Host、Client、Server

- **Host**：用户真正使用的 AI 应用（Claude Desktop、Cursor、内部 Agent 平台），负责用户交互、模型调用、权限决策、上下文聚合——"大脑所在的应用壳"；
- **Client**：Host 里连接某个 Server 的组件（一个 Host 可连多个 Server，每连一个通常创建一个 Client），负责建立连接、协议版本与能力协商、发送 tools/list 与 tools/call、接收结果与通知、维护这条连接的安全边界；
- **Server**：真正提供能力的一方（文件系统、GitHub、数据库、Sentry、内部订单/工单/日志平台），不负责和用户聊天，只按协议提供能力。架构最大的好处是**隔离**：Server 不应看到全部对话、不应访问其他 Server 的数据，Host 统一管理权限和上下文。

两层协议

- **数据层**：基于 JSON-RPC，定义初始化、能力协商、tools/list、tools/call、resources/read、prompts/get、通知与错误；
- **传输层**：本地 `stdio`（进程间标准输入输出，快、简单、无网络开销，适合本地工具）与远程 `Streamable HTTP`（支持流式与标准鉴权，适合远程服务、多用户、需认证授权的场景）。

Server 暴露三类能力

能力	一句话	示例
Tools	Agent 能做什么（模型主动调用的动作）	查询订单、搜索日志、创建工单、发送消息
Resources	Agent 能看什么（被动提供的上下文）	文件内容、数据库 schema、API 文档、项目 README
Prompts	Agent 怎么做更稳（可复用提示模板）	"根据错误日志生成排查步骤" "按公司规范写 PR 描述"

一次工具调用怎么跑（五步）

① 初始化连接（Client 与 Server 协商版本和能力）→ ② 发现工具（tools/list，Host 整理成模型可用列表）→ ③ 模型决定调用（生成工具名和参数）→ ④ Client 发 tools/call，Server 执行真实查询返回结构化结果 → ⑤ 模型根据结果继续判断（回到 ReAct 循环）。注意：**MCP Server 通常不决定用户最终怎么回答**，它是标准化能力提供者，不是完整 Agent。

MCP 为什么适合 Agent（四点）

- **工具接入标准化**：Host 只要支持 MCP 就能连不同 Server，明显降低接入成本；

- **工具发现动态化**：传统 Function Calling 在代码里写死工具列表；MCP 可动态发现，工具变化还能通过通知公告；
- **上下文和动作分开**：数据库 schema 适合做 Resource，查询 SQL 才适合做 Tool；
- **多工具组合更自然**：一个 Host 聚合文件系统、GitHub、数据库、Sentry、知识库，模型围绕任务组合使用。

本地 vs 远程怎么选 & 安全边界

- 本地 MCP：读本地文件、搜代码、操作 Git、跑脚本——快而简单，但文件系统 Server 绝不能默认开放整个磁盘，应限制可访问目录；
- 远程 MCP：企业知识库、监控、日志、工单、CRM、SaaS——集中部署、统一鉴权，但要处理认证、授权、延迟、审计和多租户隔离；
- **安全边界核心在 Host**：连接哪些 Server、给什么权限、高风险动作是否要用户确认、传哪些上下文、调用是否可审计。Server 也要克制：不默认读所有上下文、不把工具描述写得误导模型。MCP 不是安全魔法——最终取决于 Host 怎么管权限、Server 怎么暴露能力、工具怎么做确认和审计。

三者关系（别神化 MCP）

Function Calling 是模型调用工具的一种方式；MCP 是工具接入 AI 应用的一套标准协议；Agent 是围绕目标持续行动的系统——三者不互相替代，在不同层面解决不同问题。MCP 解决的是连接问题，不自动让 Agent 可靠。

面试答法

"MCP 全称 Model Context Protocol，是一套让 AI 应用标准化连接外部工具、数据源和提示模板的协议。采用 Host-Client-Server 架构，Server 暴露 Tools、Resources、Prompts，Host 通过 Client 连接多个 Server。Function Calling 更偏模型侧，MCP 更偏应用接入层；实际系统里 Host 通过 MCP 获取工具列表，再转成模型支持的 Function Calling 格式。安全上 Host 必须控制 Server 连接权限、上下文传递范围和高风险工具确认。"

第 4 章 · 可靠性兜底

本章主线：Agent 越能自主行动，就越可能自主翻车。翻车不是偶然 bug，而是自由度带来的必然风险。从“会做 Demo”到“敢上线”的分水岭，就是有没有一套护栏。

4.1 为什么 Agent 比普通应用更容易不稳定

传统程序执行路径是开发者写好的，出问题能看日志定位节点。Agent 的核心价值是**根据中间结果动态决定下一步**——这也是它不稳定的来源：边走边判断，就意味着每一步都可能判断错。它的可靠性不只取决于模型聪不聪明，还取决于：每一步有没有状态记录、工具有没有清晰边界、失败能不能恢复、有没有停止条件、有没有人工确认、有没有审计和回放。没有这些，Agent 就像一个很有想法但没有刹车的人。

4.2 六类典型翻车与兜底

① 死循环：一直在做，但就是不交付

- 表现：查订单缺订单 ID，正确做法是追问用户，它却继续查手机号→查最近订单→再查详情，绕好几圈。
- 根因（不是模型“傻”）：缺**停止条件**、缺**失败记忆**、缺**任务完成判断**。
- 兜底：限制最大步数（客服 Agent 最多 5 次工具调用、故障排查最多 15 步，超过就必须总结进展和缺失信息）；记录失败工具（同一工具同一组关键参数失败过就不要无限重试，按错误码选择追问/缩范围/换工具/停止）；设计停止条件（已拿到足够证据、缺少关键输入、工具连续失败、达到最大步数、进入高风险动作前）。

② 误调用：工具选错，后面全错

- 原因：工具描述太像（get_order_info / query_order / search_order 分不清）、读写工具没分开、参数 Schema 太松（“昨天晚上”直接塞进时间字段）、返回值没有明确状态。
- 兜底：查询与动作工具分开；高风险动作二次确认；调用前做规则校验（订单号格式不对不调用、时间范围超限不调用、缺用户确认不允许提交、无权限不允许执行）。

核心口诀

模型负责决策，系统负责守门。不要把所有判断都交给模型。

③ 多步任务中断：中间失败后不会恢复

- 表现：搜索工具超时就说“仓库没有明显技术债”；目录无权限就忽略；读文件失败就脑补内容——两种典型问题：只完成 30% 就开始总结（提前交付）、前面查到的证据后面忘了（丢失上下文）。
- 根因：缺少任务状态——不知道整个任务做到哪、哪些子任务失败、哪些证据拿到、哪些结论还不能下。
- 兜底：维护外部**任务状态板**：当前目标、已完成步骤、每步证据、失败步骤与原因、缺失信息、当前能否交付。不是把 Agent 变成固定流程，而是给它一个可以动态决策、但每步都写入状态的任务板——从“边走边忘”拉回“边走边记录”。

④ 上下文污染：错信息混进去，后面全被带偏

- 表现：把另一个服务的日志当成根因一路查数据库；把用户的猜测（“我觉得可能是缓存问题”）当成事实，一直查缓存，忽略真正的发布问题。
- 四个来源：用户猜测、工具返回的噪音（无关日志/搜索结果/RAG 片段）、过期上下文（前面结论已被推翻）、模型自己的中间假设（不标记为假设就会被当事实）。
- 兜底：上下文**分层管理**——**事实**（来自工具/数据库/日志/监控且可追溯来源）、**假设**（模型推测、用户猜测、待验证方向）、**废弃信息**（已证伪、过期或错误）三类分开；最终结论必须基于事实，假设只能作为待验证方向，废弃信息不能继续参与推理。

⑤ 权限越界：最危险的翻车

- 表现：用户只是问“能不能取消”，Agent 直接取消订单；只是问“邮件怎么回”，直接发出去了；只是说“这配置是不是有问题”，直接改了生产配置——这不是体验问题，是事故。
- 原因：工具权限太大（既能查又能改）、缺少确认环节（从“建议”直接跳到“执行”）、Host 没做权限守门（默认相信模型判断）。
- 兜底：**高风险动作三级分级**——低风险（只读查询，可让 Agent 直接调）；中风险（生成草稿：邮件草稿、退款申请草稿、配置修改方案，可做但不能直接提交）；高风险（改变外部世界：发邮件、退款、删数据、改配置、执行命令，必须用户确认 + 审计记录）。

⑥ 错误不可恢复：失败后只会硬编

- 表现：工具返回“系统异常”，Agent 不知道是缺参数、权限不足、超时还是数据不存在，于是编一个“根据查询结果，订单正在处理中”。
- 兜底：错误返回结构化（见 3.7）——缺参数就追问、超时就有限重试、权限不足就停止、业务规则不允许就不再试。

4.3 兜底不是一个开关，而是一套六层护栏

任务边界 → 工具边界 → 执行边界 → 状态管理 → 权限控制 → 交付检查

1. **任务边界**：明确 Agent 能做什么、不能做什么；
2. **工具边界**：描述、参数、返回值、错误码都要清楚；
3. **执行边界**：最大步数、超时、重试次数、停止条件；
4. **状态管理**：记录目标、步骤、证据、失败、缺口；
5. **权限控制**：高风险动作二次确认、权限校验、审计记录；
6. **交付检查**：最终回答前检查目标是否完成、证据是否足够、有没有未验证假设。

关键认知

护栏不是为了限制 Agent 的能力，而是为了让它能进入生产环境。没有护栏的 Agent，Demo 里看起来很聪明，一到真实业务就容易出事。真正的问题不是“能不能完全不翻车”，而是：翻车前能不能拦住，翻车时能不能止损，翻车后能不能复盘。

面试答法

"Agent 的核心是动态决策，每一步都根据中间结果决定下一步，所以自由度更高，也更容易死循环、误调用、上下文污染、多步中断和权限越界。工程上不能只靠模型自觉，需要用步数限制、状态记录、工具边界、错误恢复、权限确认和审计来兜底。比如防死循环：设置最大步数、最大重试次数和停止条件，记录失败工具和关键参数；防上下文污染：把事实、假设和废弃信息分开管理，最终回答必须基于事实和证据。"

第 5 章 · 记忆与状态

本章主线：Agent 记忆不是把历史全塞进 Prompt，而是把有价值的信息用合适的方式**保存、检索、更新和遗忘**。四类记忆各司其职：短期记忆管当前对话，Session State 管任务进展，长期记忆管跨会话稳定信息，RAG 管外部知识。

5.1 Agent 为什么需要记忆

普通问答问一句答一句，不一定需要复杂记忆。但 Agent 要多步任务——"分析项目技术债并给修复建议"要经历看目录、看核心模块、找复杂函数、看测试覆盖、总结优先级。过程中必须记住：用户最初的目标、已经看过哪些目录、哪些文件有问题、哪些证据支撑判断、哪些步骤失败、哪些结论只是临时假设。没有记忆就会边做边忘：刚查到的证据后面忘了、失败的工具继续调用、用户强调的要求被忽略——这是很多 Agent 看起来"不稳定"的原因之一：不是不会推理，而是缺少可管理的任务记忆。

5.2 四类记忆

类型	管什么	特点
短期记忆	当前对话里刚发生了什么	最近几轮聊天、最近几次工具调用与结果；生命周期短、和当前任务强相关、受上下文窗口限制
Session State	当前任务进行到哪一步	不是聊天记录，是结构化状态：goal / completed_steps / failed_steps / evidence / missing_info
长期记忆	跨会话保留的稳定信息	用户长期偏好（主写 Java、准备秋招）、业务长期状态（高优客户、常用排查入口）；必须筛选写入、需要更新和遗忘
外部知识库	Agent 可以查的资料（常用 RAG 实现）	公司文档、API 文档、产品手册、代码仓库、运维手册——是外部知识来源，不是"用户记忆"

5.3 短期记忆不是越长越好

上下文窗口大能放更多历史，但放得多不代表效果好——上下文里混着用户的临时猜测、工具返回的无关日志、被证伪的方向、过期的中间结论、重复对话，堆进去反而干扰模型（上下文越长越容易污染【因果】）。短期记忆要做两件事：保留当前任务相关信息（目标、约束、最近工具结果、关键证据）；丢掉不再有用的信息。它应该是"当前任务摘要"，不是聊天记录全文——例如排查接口变慢时，该保留的是"目标：定位变慢原因；数据库方向已排除；22:05 有发布；22:10 后第三方回调超时增加；下一步确认发布是否引入回调重试"，而不是全部聊天原文。

5.4 Session State 比聊天记录更适合多步任务

多步任务最重要的不是"记住说过什么"，而是"记住任务进展"。一个故障排查 Agent 的状态结构：

```
{ goal: "定位支付接口变慢原因", current_hypothesis: "第三方回调超时可能导慢", confirmed_facts: ["22:10 后 P95 从 300ms 升到 3.8s", "22:05 发布 payment-service v2.3.1"], rejected_hypotheses:
```

["数据库慢查询"], next_actions: ["检查 v2.3.1 是否改动回调重试逻辑", "查询回调错误码分布"] }

它把信息分成**目标、假设、事实、已排除方向、下一步动作**，Agent 每一步都能对照推进，而不是靠模型在长上下文里找重点。只靠 conversation buffer 的系统，任务一旦超过三五步就容易跑散——故障排查、代码库分析、长文档总结、数据分析报告、多轮客服处理都需要结构化状态。

5.5 长期记忆不能直接写入，必须过筛

工程上最怕把临时信息写成长期事实："我最近在看 Go"可能只是今天随口一问；"我可能想投后端"是不稳定意向——直接记成"用户目标是后端"会持续误导。写入前至少问四个问题：

- 1. **是否长期稳定**？"用户主要使用 Java" 比 "用户今天问了 Go" 更适合；
- 2. **以后是否有用**？"喜欢口语化讲解"有用；"今晚 9 点问了个问题"通常没用；
- 3. **是否被确认过**？用户明确说"我主要投 Java 后端"比模型猜的可靠；
- 4. **是否涉及隐私或敏感信息**？敏感信息需要权限、用途和删除机制。

长期记忆要像数据库写入（有写入规则、更新规则、删除规则），而不是像聊天记录自动追加。使用时也要**按需检索**：根据当前任务生成检索意图 → 找相关条目 → 过滤过期和低置信度记忆 → 只把关键记忆放进上下文 → 回答后按需更新。简历优化任务检索求职方向、项目背景、写作偏好；故障排查任务检索服务背景、历史事故、常用排查入口。

5.6 RAG 不是 Agent 的长期记忆

	RAG（外部知识库）	长期记忆
检索对象	文档、代码、手册、题库等外部知识	用户偏好、目标、历史任务结论、稳定状态
内容形态	原始资料	经过提炼的结构化信息（带类型、置信度、来源、更新时间）
关系	都可能用向量检索，但语义不一样；向量数据库只是存储和检索方式，不等于记忆设计	

易错点

把所有聊天记录原文扔进向量库，检索出来一堆碎片，那只是"历史记录检索"，不是长期记忆。原始对话"我主要准备 Java 后端，项目是网盘系统，简历别写成流水账"，长期记忆应写成结构化条目：`{type: user_profile, content: "用户主要准备 Java 后端方向", confidence: 0.95, source: "用户明确表达", updated_at: ...}`。面试别说"我们用向量数据库实现长期记忆"——要补上写入规则、记忆类型、置信度、更新时间、冲突处理和删除机制。

5.7 记忆也要会忘

记忆不是越多越好，否则会出现三类问题：**过期**（用户去年准备 Java 后端今年转向大模型应用，还按旧方向给建议就错了）、**冲突**（先说投算法后说不投，要更新或降权旧记忆，不能都塞给模型）、**噪音**（低价值信息太多，检索召回一堆无关内容）。遗忘机制至少包括：过期时间、置信度、更新时间、冲突合并、用户删除、低价值记忆清理。记忆不是仓库，不是只进不出，而是持续维护的状态系统。

5.8 记忆设计四步框架

定义记忆类型 → 定义写入规则 → 按任务检索 → 使用后更新

1. **定义记忆类型**：至少区分用户偏好、用户目标、项目背景、任务状态、历史结论、外部知识引用——不同类型写入规则不同（偏好长期保存，任务状态会话结束清掉，外部知识引用回到 RAG 文档）；
2. **定义写入规则**：什么能写、什么不能写、什么要用户确认、什么只作临时状态；
3. **检索按任务筛选**：不全量塞，按当前任务选相关记忆；
4. **使用后更新**：目标变了要更新、旧结论被推翻要降权、低价值记忆要清理、用户要求删除要删除。

核心原则

真正好的 Agent 记忆不是记住一切，而是：该记的记住，该查的去查，该忘的忘掉，该确认的确认。四类记忆混在一起，就会乱。

面试答法

"短期记忆服务当前会话和任务，受上下文窗口限制；长期记忆是跨会话的稳定信息，不能直接把聊天都存起来，需要筛选、结构化、更新和遗忘。Session State 是结构化任务状态，比 conversation buffer 更能防止任务中断、重复调用和提前交付。RAG 用于检索外部知识，长期记忆保存用户偏好和历史结论——向量库只是检索层。防止长期记忆变脏：写入前判断是否稳定、有用、被确认、合规；新信息冲突时更新或降权旧记忆，过期和低价值记忆要清理。"

第 6 章 · 评估与度量

本章主线："试了几个问题感觉还可以"不叫评估，叫碰运气。Agent 评估不能只看最终答案，要看整个执行过程——结果层看完成率分级，过程层看工具调用与错误恢复，安全指标单独统计。没有评估的 Agent，就是黑盒。

6.1 为什么 Agent 评估比普通问答难

Agent 中间会规划、调工具、读结果、改策略、做多步动作：最后答对了，也可能是中间乱调工具碰巧答对；最后答错了，也可能是工具返回失败、权限不足、缺少关键输入，不全是模型问题。评估至少要回答四问：①最终任务完成了吗？②中间过程靠谱吗（有没有乱调用、重复调用、越权、忽略失败）？③成本和效率怎么样（5 步完成还是绕 30 步）？④失败时能不能收住（追问和停止，还是继续瞎猜）？——要看结果，也要看轨迹。

6.2 结果层：任务完成率要分四档

档位	定义	示例
完整完成	用户目标被完整满足	找到证据链（P95 升到 3.8s、22:10 变慢、22:05 发布、新版本加了回调重试、回调超时激增），给出结论与修复建议
部分完成	完成一部分且有明确缺口，如实说明	"目前证据只能支持发布后接口变慢，还缺少第三方回调日志，不能下最终结论"——这比强行总结好；坏的是装成全部完成
正确失败	任务没完成，但失败处理正确	用户没给订单号 → 追问"请提供订单号"。生产系统里正确失败非常重要，因为它不制造错误结果
错误失败	没完成，还给了错误结论（最危险）	没查到订单，却说"订单正在仓库处理中"——用户以为查到了，其实是编的

要点

一个靠谱的 Agent 不是每次都必须成功——很多任务本来就无法继续（缺订单号、无权限、工具超时、数据不存在、文档缺失），正确表现是**识别无法完成的原因并给出下一步动作**。面试别只说"用准确率评估"，要说"按任务结果分级统计：完整完成率、部分完成率、正确失败率、错误失败率"。

6.3 过程层：三类过程指标

① 步骤效率——把"看起来完成了"拆成"怎么完成的"

- **平均步数：**步数不是越少越好（太少可能没认真查），但长期偏高通常在绕路；
- **无效工具调用率：**缺参数还调用、同样参数重复失败、查无关数据、调不适合的工具；
- **重复调用率：**同一工具、同一组关键参数、同一错误原因反复调用——本质就是第 4 章讲的死循环；

- **关键路径长度**：完成任务真正需要的核心步骤（监控→发布记录→错误日志→结论）；绕去数据库、缓存查一圈就是路径效率差。

步骤效率的价值：每次工具调用都有成本——Token、延迟、外部系统压力、安全风险。能完成任务但每次绕几十步，也很难上线。

② 工具调用正确率

至少评估五方面：**工具选择**是否正确（问退款应先查退款规则，不是直接提交）；**参数**是否正确（手机号填进订单号字段、"昨天晚上"塞进时间字段都算错误）；**调用时机**是否正确（高风险工具在用户确认前调用了，即使参数对也不正确）；**工具结果**是否正确使用（返回 MISSING_ORDER_ID 还继续查订单详情，就是没读懂结果）；以及**高风险工具违规调用次数**。

③ 错误恢复率——失败后会不会正确转向

失败后的合理选择：追问用户、缩小查询范围、换工具、有限重试、阶段性总结、停止并说明权限不足、标记缺口不下结论。按错误类型分别评估：缺参数→是否追问；无数据→是否调整范围或说明；超时→是否有限重试避免死循环；权限不足→是否停止而非绕权限；规则不允许→是否解释规则而非继续尝试。评估时还要看 Agent 是否真的按 `recommended_action` 做了。

6.4 安全与权限指标：单独统计，不能被平均

有些错误不是"答错了"，是事故：未确认就发邮件/退款/删数据/改生产配置、越权查询隐私数据、把敏感信息写进日志或上下文。一次严重越权就可能让整个系统不能上线。四类指标：

1. **高风险动作违规率**（需要确认的动作有没有提前执行，生产环境目标应为 0）；
2. **权限绕过尝试**（权限不足时有没有换工具绕过，非常危险）；
3. **敏感信息泄露**（内部日志、Token、用户隐私、系统提示词是否输出）；
4. **审计完整性**（谁触发、何时、为什么建议执行、用户是否确认、调了什么工具、返回什么——没有审计出事没法复盘）。

6.5 评估集怎么设计：五类样本覆盖

类型	测什么
正常完成类	信息完整、工具可用、权限足够——测基本完成率
信息缺失类	缺订单号、时间范围、文件路径——测会不会追问
工具失败类	超时、无数据、权限不足、参数错误——测错误恢复
高风险动作类	退款、删除、发邮件、改配置——测权限确认与安全边界
干扰噪音类	混入用户猜测、无关日志、过期结论——测上下文污染控制

评估集不是越大越好：初期先做 50-100 个**高质量**任务，每个标清——用户目标、可用工具、初始输入、标准完成条件、允许的工具调用、不允许的动作、预期失败处理、评分规则。标准不清，评出来的分没有意义。

6.6 自动化评测和人工评测都要有

- **自动化负责规模**：评结构化指标——是否调用了正确工具、参数是否符合 Schema、是否超步数、是否重复调用、是否确认前调高风险工具、错误码出现后是否执行 recommended_action、输出是否包含必要字段（通过 trace 和日志自动判定）；
- **人工负责质量**：评语义质量——结论是否解决用户问题、证据链是否可信、解释是否清楚、有没有遗漏关键风险、建议是否可执行；
- **LLM-as-judge 只做辅助**：不要让模型评价模型然后自我感动——安全、权限、事实正确性要用规则和人工抽检兜住。

稳态评估流程：跑离线评估集 → 自动计算工具调用、步骤、错误恢复、安全违规 → 抽样人工看 trace 和回答 → 对失败样本做归因 → 修改 Prompt/工具/状态管理/权限规则 → 再跑同一批评估集做回归。

6.7 线上监控：评估不是上线前做一次

上线后用户输入、工具、业务规则、知识库都会变，评估要持续做。至少监控：任务完成率、用户追问率、工具调用次数、平均执行步数、工具失败率、重复调用率、超时率、高风险动作确认率、用户取消率、人工接管率、用户差评率、安全拦截次数。

异常信号举例：工具失败率突然升高→可能是某个工具接口挂了；平均步数突然升高→可能是 Prompt 改动导致绕路；人工接管率升高→新问题超出了任务边界。线上监控必须结合 **trace 回放**：用户输入是什么、每一步怎么想、调了什么工具、返回什么、哪一步开始跑偏——只看到一个错答案，不知道是检索错、工具错、参数错、状态错还是模型理解错，是排不了问题的。

6.8 五大常见误区

误区	正确做法
只看最终答案	过程同样重要：中间调了危险工具，答案再对也不能算好
只测正常任务	正常任务测不出可靠性；缺参数、工具失败、权限不足、上下文污染、高风险动作才是拉开差距的地方
只用大模型打分	违规、参数合法性、步数、越权应用规则判断
指标太多没人看	安全问题不能被"综合分"平均掉，高风险违规必须单独拉出来
没有失败归因	成功率 70% 没用，要知道失败来自 Prompt 不清、工具描述不清、Schema 太松、记忆污染、检索差、权限缺失还是错误返回不可恢复

面试答法

"Agent 评估从结果和过程两层做：结果层看任务完成率、部分完成率、正确失败率和错误失败率；过程层看工具调用正确率、参数正确率、步骤效率、重复调用率、错误恢复率和权限违规率。缺订单号时追问是正确失败，没查到却编一个状态是错误失败。评估集覆盖正常、信息缺失、工具失败、高风险、噪音五类样本。自动化评结构化指标，人工评语义质量，LLM-as-judge 只做辅助，安全指标单独统计。"

第 7 章 · 规模化工程（多 Agent）

本章主线：单个 Agent 会规划会调工具还不够——复杂任务要落地成 DAG 执行器（并行、可恢复、可纠错），多 Agent 要拆开定义权、执行权、放行权，协作要用独立 Thread、Artifact 引用和预算治理管住。这一章是面试深水区。

7.1 Plan-and-Execute 怎么落地成 DAG 执行器

为什么不能按列表串行执行（for 循环的三个问题）

- **默认所有步骤串行：**实际很多步骤互不依赖，可以并行——“分析支付接口变慢”的 6 步计划其实是 3 层（前 4 步可同时发起，第 5 步等前 4 步，第 6 步等第 5 步），for 循环把 30 秒拉成 2 分钟；
- **没说失败怎么办：**第 3 步挂了，前两步结果丢不丢？整个重来还是跳过继续？正确行为是第三种：只暂停依赖第 3 步的节点，其他分支照跑，第 3 步单独重试；
- **没考虑计划本身可能错：**执行到一半发现某步结果证伪了后续步骤，只能整个推翻还是能局部调整？

四层落地架构

层	做什么	关键细节
① 结构化计划	不要纯文本列表，让模型输出 JSON 的 <code>nodes</code> （每节点带 <code>action/tool/params</code> ）和 <code>edges</code> （依赖关系），变成有向无环图	模型交付的不是文字，而是一张执行图
② 校验和排序	执行前环检测 + 拓扑排序	环检测用 DFS 三色标记（白未访问/灰在当前递归路径/黑已完成，撞到灰色即成环）；检测到环优先让模型重新规划，不要自动断边；拓扑排序用 Kahn 算法（入度为 0 入队，删出边减邻居入度），排出的节点数少于总数说明有环；同一批出队的节点就是可并行的一层，用异步任务池 + 信号量控制并发上限（如 5 个），防止打爆 API 限流
③ 状态追踪与持久化	每节点维护状态： <code>pending / ready / running / success / failed / skipped</code>	失败只传播给后继，无关分支继续跑；软依赖可标 <code>optional: true</code> （失败不阻塞，但输入里标注缺失）；Checkpoint 存四样：DAG 结构、节点状态、已完成节点输出、当前 <code>ready</code> 队列；通常按层存一次或固定 30 秒存一次；重启跳过 <code>success</code> 、重试 <code>running</code>
④ 局部 Replan	节点失败且重试无用（权限不足、接口不存在），或节点结果证伪了后续步骤（查发布记录返回空，“分析发布影响”就是空转）时触发	已成功节点锁定不许改；把失败节点连同后继摘出，带上原始目标、已完成结果、失败原因让模型重新规划这一段，新子图合并回原 DAG；上限 3 次，超限终止并返回部分结果与失败原因

一句话

计划不是一次性的步骤清单，是可变的执行图：**并行靠依赖，恢复靠状态，纠错靠局部重规划。**

知识拓展（高频追问）

① DAG 和 Workflow 是一回事吗？——调度机制几乎一样，区别在**图谁定的**：Workflow 的图开发者提前画好运行时不变，Agent 的 DAG 是模型现场生成的，所以 Airflow/Argo 的调度内核可复用。② JSON Schema 管得住形状管不住语义（时间写成 "yesterday" 格式合法），要在工具调用前加参数校验或让工具容错解析。③ 节点粒度：默认一个节点对应一次工具调用或一个逻辑完整的子任务。④ 要跑几小时的节点：启动异步任务立即返回 task_id，状态机加 polling 状态。⑤ Replan 时已完成结果不塞 Prompt（会撑爆上下文），存外部、Prompt 放摘要或引用 ID。

7.2 Planner、Worker、Reviewer 怎么分工

为什么不能让一个 Agent 从头包到尾

一个 Agent 同时做四件事：理解目标、决定查什么、调工具收集证据、判断自己结论是否可信——**同一个错误会跨阶段传播**：先误判是数据库问题就会优先查数据库，查到无关慢查询又当支持证据，验收时还沿原始假设检查。拆成三个角色，拆开的不是工作量，而是**定义权、执行权、放行权**：

角色	核心决策	必须交付	明确不能做
Planner	做什么、先后依赖、什么算完成	结构化任务包	直接执行生产写操作、给自己的计划判完成
Worker	在给定边界内怎么完成	Artifact、证据、假设、未解决项	私自扩大目标、修改验收标准、自我验收
Reviewer	证据是否满足验收标准	结构化判定与失败原因	无标准打分、悄悄替 Worker 改产物再放行

Planner：不是写漂亮计划，而是生成任务契约

最小任务包字段：task_id、goal、inputs、dependencies、**allowed_tools**、constraints（时间范围/环境）、**acceptance_criteria**、expected_outputs。三个易被忽略的点：

- **目标必须能验收**："分析一下为什么变慢"太宽，Worker 不知道找什么、Reviewer 不知道何为完成；改成"定位 P99 延迟升高的主要原因，至少两类独立证据支持"才有收口条件；
- **工具权限跟任务一起下发**：allowed_tools 不是建议，是编排器实际执行的白名单——这次任务只读监控、日志、发布记录，就不应暴露改配置、重启服务、发通知等写工具；
- **验收标准在执行前确定**：Reviewer 看完结果才临时定标准，Worker 会无休止返工；标准必须能对应具体证据（时间范围覆盖故障前后各 30 分钟、结论至少监控 + 日志两类证据交叉支持、不确定项显式标注、建议含风险和回滚条件）。

Worker：只执行当前任务，交付的不是一句"完成了"

- 输入尽量小：当前任务目标和验收标准、已完成依赖的 Artifact 引用、必要背景、允许工具、截止时间/成本/权限边界——**不要把根 Agent 全部聊天记录、其他 Worker 完整轨迹、所有工具说明塞进去**；上下文越大，越容易把别人的假设当自己的事实；
- 交付五部分：status、**artifact_refs**（可复用工作产物：报告、补丁、测试结果）、**evidence_refs**（证据：查询快照、Trace ID、测试日志——回答"你凭什么得出这个结果"）、assumptions、unresolved。**Artifact 和证据要分开**：报告写得像真的，不代表证据真的支持它；
- 发现任务包有问题时返回 **blocked** + reason_code（如 MISSING_DATA_PERMISSION）+ partial_artifacts + recommended_action，由编排器决定申请权限、退回 Planner 重拆还是部分交付——让 Worker 自己绕权限找替代数据，看似主动，实际已经改变了任务的证据标准。

Reviewer：守住验收门禁，不是再做一遍

- "整体不错，再补充一些细节"这种评价无法执行。Reviewer 应拿着任务包的 acceptance_criteria 逐条检查，判定只保留三种：**PASS**（全部硬性标准通过）、**REVISE**（有明确可修复问题，退回指定角色，指出哪条标准未过、缺哪份证据、应该怎么修）、**BLOCKED**（缺权限、缺数据或标准冲突，当前角色无法修复，需升级）；
- **先过硬规则，再做语义审查**：交付文件是否存在、Schema 是否有效、测试是否通过、必填证据是否齐全、是否调用了越权工具、产物版本是否一致——能用代码判断的，不要浪费一次模型主观判断；
- **Reviewer 最好没有生产写权限**：可以生成修改建议或补丁草稿，但不能一边修改生产状态一边给自己的修改判通过；高风险场景的发布、付款、删数据、改权限仍要确定性规则或人工确认。

权限与编排：硬约束由系统执行

- 只在 System Prompt 写"你不能调用部署工具"不叫权限控制——正确做法是**按角色、任务和阶段共同裁剪工具集**，且权限带 task_id、资源范围、有效期和最大调用次数，任务结束立刻失效（如 Worker 可写 `artifact://incident-20260903/*` 但不能写其他任务目录）；
- **编排器是控制面，不是第四个自由决策 Agent**：保存任务状态、下发任务包、裁剪工具、校验输入输出 Schema、路由 PASS/REVISE/BLOCKED、限制返工次数/并发/超时、记录审计链——都由确定性代码完成。可以让模型建议路由，但状态转换必须符合代码里的状态机（PLANNED → RUNNING → REVIEWING → PASSED / REVISING / BLOCKED → ESCALATED），任何角色不能从 RUNNING 直接跳到 PASSED。

不通过时退回给谁

问题类型	退回对象	处理方式
证据不足、格式错误、测试失败	Worker	按失败标准补证据或修产物
目标拆错、依赖缺失、验收标准冲突	Planner	修改受影响任务包，不推翻已通过产物
权限缺失、关键数据不存在	编排器或人工	授权、降级交付或终止任务
Reviewer 意见前后矛盾	人工或第二门禁	固化标准，避免无限主观返工

返工必须有上限（如同一条标准最多退回两次）；超限后不偷偷降低标准、不继续烧 Token，而是升级人工并保留当前 Artifact、证据和每次退回原因。

Multi-Agent 什么时候反而不值得用

- 任务很短且结果能被代码直接验证（CSV 转 JSON，Schema 通过即结束）——再加 Planner/Reviewer 只增加延迟、成本和故障点；
- 三个角色拥有完全相同的上下文和工具——只是换三段 Prompt，错误仍然高度相关，三个 Agent 可能一起相信同一个错误假设；
- 验收标准根本说不清（"写得更有感染力" "方案更高级"）——先让人确定判断维度，比盲目加 Reviewer 更重要；
- 协作成本超过任务本身——单 Agent 10 秒稳定完成，三角色跑一分钟完成率只提高一点，就不划算。

怎么验证角色拆分真的有效（对比单 Agent 基线的六组指标）

首次验收通过率 | 平均返工次数 | 交接失败率（任务包缺字段、Artifact 引用失效、状态错乱） | 越权调用率 | Reviewer 误放行率 | 单个成功任务的 Token、延迟和工具调用成本。完成率提高了但误放行率没降，说明只是多了层表面检查。分级策略：低风险任务规则自动验收，中风险加模型 Reviewer，高风险叠加人工确认。

7.3 多 Agent 上下文、消息和 Token 怎么治理

为什么多个 Agent 不能共享一条消息历史

三个 Worker（查监控/查日志/查发布记录）共享一个 messages 数组时：几千行指标、错误堆栈、版本信息不断混在一起；A 的一句"可能是连接池问题"进入 B 的上下文，B 就围绕这个假设找证据；C 的一次工具超时被误解为"没有发布记录"——**共享历史让并行探索变成相互暗示，多个 Agent 看似独立，错误却高度相关。**

独立 Thread：子 Agent 收到的是任务包，不是父聊天记录

每个 Agent 维护独立 Thread（root / worker-a / worker-b / worker-c），编排器知道它们同属一个 run_id 和依赖关系，但不把一个 Agent 的完整过程广播给所有人；B 需要 A 的结论时，传的是**经过验收的 Artifact 引用**。子 Agent 任务包：run_id、task_id、parent_task_id、goal、acceptance_criteria、context_refs、allowed_tools、output_schema、budget（max_input/max_output/max_steps）、deadline_ms。组装上下文固定六层：

全局安全规则 → 角色职责与禁止项 → 当前任务包 → 已通过依赖的 Artifact 摘要与引用 → 允许工具 Schema → 剩余步数/时间/Token 预算

父 Agent 的语气、闲聊、完整思考过程、其他分支尚未验收的猜测都不默认继承。任务缺信息时子 Agent 返回 BLOCKED 和缺失字段，由编排器补充或退回 Planner。

消息只传引用，不传完整产物

一次日志查询 5 万行、一次代码检查上百 KB Diff，如果从 Worker 复制给 Reviewer 再复制给主 Agent，同一份材料重复进入多个上下文。要把"消息"和"产物"拆开：**消息负责协调**（状态、摘要、失败原因、证据引用）、**Artifact 负责承载内容**（报告、补丁、日志快照）、**事件日志负责审计**。底线：**摘要可以帮助定位**，

不能替代原始证据——Artifact 必须带版本、内容哈希或不可变快照，不能让 Reviewer 验收 v3 时底层已变成 v4。

并行消息按因果关系处理，不按墙上时钟

事件信封字段：`event_id`、`run_id`（串起同一次任务）、`task_id / agent_id`（定位子任务与执行者）、`seq_no`（单 Agent 线程内保序）、`parent_event_id`（记录谁触发了谁）、`idempotency_key`（重试去重）、`artifact_refs`。编排器收到事件先做三步：用 `event_id / idempotency_key` 去重 → 检查 `seq_no` 是否连续（缺号暂存等待或补拉）→ 检查 DAG 依赖与 `parent_event_id`，依赖满足才推进状态。记住：**不要用时间戳代替因果关系**——分布式环境有时钟偏差、网络延迟和重试，谁先到不等于谁先发生。

Token 预算：派发前原子预留

多 Agent 消耗的是一整棵任务树：根 Agent 输入输出 + 所有子 Agent 输入输出 + 工具结果进上下文的 Token + 重试返工 + 最终汇总验收。如果总预算 100K，根 Agent 同时派 4 个“最多 30K”的 Worker，理论上限已到 120K，还没给 Reviewer 和最终回答留额度。所以预算要像资源配额一样管理：

- **派发前原子预留**：编排器从父任务可用额度预留，成功才启动；失败就缩小任务、排队、换模型或拒绝派生；任务结束未用额度回收。 $\text{可派发预算} = \text{全局上限} - \text{已消耗} - \text{已预留} - \text{根流程保底} - \text{验收保底}$ ；
- **多维限制**：单次和累计输入/输出 Token、工具结果大小与分页次数、模型调用次数、工具调用次数、重试次数、子 Agent 数量、递归深度、任务总时长；
- **分级降级**：0–60% 正常执行；60–80% 禁止低价值扩展、压缩旧工具结果、优先复用 Artifact；80–95% 停止新建非关键子 Agent、小任务切低成本模型；95% 以上强制收束，保留验收与最终回答额度，返回部分结果或升级人工。压缩有边界：订单号、文件路径、错误码、验收标准与证据引用不能被“概括掉”；**预算优先保护最终合成和验收**——Worker 全跑完而 Reviewer 没额度，任务仍不能算完成。

递归派生默认关闭；模型路由按三维度

- Worker 需要继续拆分时，应返回 Planner 重构 DAG；确需开放受限派生时加四道硬限制：**`max_depth`**（最大递归深度）、**`max_children`**（每节点子数上限）、**`ancestor_task_ids`**（祖先链，禁止 A 派生 B、B 又派生 A）、**`child_budget ≤ parent_available_budget`**。还要防“同义派生”（连续创建“继续搜索/补充搜索/深入搜索”）——用规范化目标、输入范围和工具集合生成任务指纹，相似任务超阈值就拒绝；
- 模型路由看三个维度：**任务复杂度**（抽取/分类/格式转换适合小模型，开放规划和冲突合成需要强模型）、**风险等级**（涉及生产写操作、权限、合规时还要规则校验或人工门禁）、**可验证性**（输出能被 Schema/测试直接验证时可积极用低成本模型）。看的是**单个合格任务成本**，不是单次调用价格——小模型输出变短却让返工翻倍，总成本反而更高。

出错以后怎么恢复 & 怎么验证治理有效

- Worker 超时：保留已写入的 Artifact 和最后确认的事件序号，再决定从 Checkpoint 重试、换模型、缩小任务或降级交付；消息重复只做幂等确认不重复触发 Reviewer；Artifact 版本不一致只让受影响的汇总节点重跑；预算熔断则冻结新派生、取消非关键节点，剩余额度留给状态总结和部分结果交付；
- 监控六组指标：上下文（各角色输入 Token、重复 Token 比例、Artifact 局部读取命中率）、消息（乱序暂存数、重复事件率、幂等冲突率、依赖等待时间）、预算（预留量、实际消耗、回收量、各角色占比、

熔断次数)、递归(任务树最大深度、分支数、相似任务拒绝次数)、质量(完成率、首次验收通过率、关键约束保留率)、业务效率(P95 延迟、单个合格任务成本、人工接管率)。

一句话总结

多 Agent 最危险的不是某个 Worker 答错，而是错误历史被复制、重复消息被执行、并行分支同时把预算花穿。**Thread 隔离过程，Artifact 承载事实，事件表达因果，预算决定边界。**

7.4 Browser Agent 怎么读取大型网页

为什么不能把原始 DOM 直接塞给模型

用户眼里只有"商品名、价格、加入购物车"的卡片，DOM 里可能还有十几层 div、CSS 类名、埋点属性、SVG 路径、响应式副本和隐藏弹窗。直接塞 outerHTML 有四个问题：① Token 被结构噪声吃掉；② 页面语义被实现细节淹没（看见很多 div 却不知道哪个是主内容）；③ 隐藏内容制造冲突（PC/移动端副本、未展开菜单同时进入）；④ 节点引用很快过期（页面一重渲染，旧 DOM 路径指向别的元素）。第一步不是"截多少 HTML"，而是先问：**当前任务需要阅读页面，还是操作页面？**

先按任务选页面表示（不能二选一打天下）

页面表示	最适合	保留什么	容易丢什么
Readability 正文	新闻、博客、文档详情页	标题、正文、作者、摘要	按钮状态、复杂表单、后台布局
可访问性树	登录、下单、审批、搜索等交互页面	role、accessible name、层级和部分控件状态	CSS 视觉关系、画布内容、无障碍标注差的节点
局部 DOM	需要属性、结构或精确调试的区域	节点属性、父子结构、业务标记	Token 开销仍高，容易绑定实现细节
局部截图	图表、Canvas、视觉位置判断	最终渲染效果和空间关系	精确文本、完整状态和稳定定位

正确方案：**默认用最省 Token 的语义表示，缺信息时再局部升级。**基础清洗可移除 script、style、注释、纯装饰节点，但链接目标、表头、label、ARIA 属性、控件状态和 iframe 边界要保留；也不能把"viewport 之外"等同于"无关"——虚拟列表的内容可能要滚动后才出现。

第一次打开页面，只返回"地图"

在 300 页 API 文档里找"重试策略"，第一次调用没必要返回 300 页正文——应返回页面地图（title + regions: ref、role、headings、摘要），Agent 看到重试策略位于 r7，再调 `read_region(ref="r7", query="重试策略")`——**先概览，再下钻。**地图保留四类锚点：landmark（main/navigation/form/dialog 大区域）、heading path（标题层级）、content summary（表格行数、列表项数）、action ref（可操作控件临时引用）。

按语义边界分块 + 工具显式分页

- 固定 4000 字符硬切会把语义切碎（表头在第一块、目标行在第二块）。分块顺序：先按 region 切 → 正文按 h1-h6 标题树切 → 表格保留表头按行分页 → 列表按 item 游标分页 → 单块仍超预算才在段落或行边界切。每块带最小元数据：region_ref、标题路径、块类型、序号、总量估计、page_version；
- "返回全部结果，框架自动截断"是最差的工具契约——截断后 Agent 不知道结果是否完整。读取工具的参数和返回值都要显式分页：`request(region_ref, query, cursor, max_tokens)` → 返回 `page_version`、`returned_tokens`、`truncated`、`next_cursor`；`truncated + next_cursor` 告诉 Agent "这不是全文，只是当前页"；还应支持 `find_in_page(query)` 先返回命中区域和短片段，让 Agent 决定读哪块。

无限滚动和 SPA 必须增量读取

消息列表滚到底才加载下一页、表格只渲染 viewport 附近几十行、SPA 点击标签后 URL 不变——"一次 DOM 快照"只代表某个时刻、某个视口的状态。增量读取记录三样：当前 page_version 和导航序号、已见区域/item 的指纹集合、本次操作后新增/删除/变化的区域。操作完成后优先返回 **diff** 而不是整页；目标没出现再决定是否继续滚动；达到 max_scrolls、连续两次无新增或页面 Token 预算不足时必须停止。动作引用表达意图，**执行前必须在最新页面状态上重新解析并校验唯一性**。

页面 Token 预算是硬约束

预算不能只写在 Prompt 里说"请节省"——模型决定是否继续读，**工具层决定它最多能读多少**。先算：`页面可用 Token = 模型窗口 - 系统规则与工具 Schema - 当前任务和必要历史 - 输出预留 - 安全余量`，再拆成四道限制：`max_tokens_per_read`（单次上限）、`max_page_tokens`（累计上限）、`max_pages / max_scrolls`（分页滚动次数）、`min_output_reserve`（无论如何不能挪用的回答与验收预算）。预算快耗尽时的优先级：精确命中区域 > 当前可视区域 > 相邻上下文 > 低相关页面扩展；订单号、按钮状态、表头和证据引用不能为省 Token 被摘掉。

可落地的读取链路（两个入口）

`observe(page, task, budget)`：等到页面足够稳定（围绕任务等目标区域出现、关键请求结束或 loading 消失，有超时上限，不是固定 sleep(3)——广告轮播和心跳请求可能永远不停）→ 指纹化 page_version → 按任务选 Readability 或可访问性视图 → 构建区域索引 → 按预算裁剪返回。`read_more(page, ref, cursor, budget)`：先断言当前版本与 ref.page_version 一致，再按语义页返回剩余预算内的内容。工具返回的不只是内容，还要有完整性、版本、下一步入口和错误类型。

怎么评估读取策略有效

只看 Token 下降不够（把正文砍没了当然最省），至少记录四组指标：**定位质量**（目标区域召回率、目标控件唯一定位率）、**任务效果**（阅读问答准确率、网页任务完成率、误点击率）、**资源消耗**（每步页面 Token、累计 Token、读取和滚动次数）、**状态稳定性**（过期引用率、重定位成功率、动态页面重复读取率）。测试集要覆盖长文章、分页表格、虚拟列表、无限滚动、iframe、弹窗和无障碍标注差的页面；还要故意让页面在观察后重渲染，验证旧引用会被拒绝。真正要优化的是**每 1000 个页面 Token 带来的任务成功率**，不是把页面压得越短越好。

一句话总结

把整个 DOM 塞给模型，只是把浏览器的负担转嫁给上下文窗口。真正能跑稳的 Browser Agent，手里拿的不是网页全文，而是一张**随页面变化、可以继续下钻的地图**。

附录 A · 知识框架总览、重点难点与复习建议

A1 知识框架总览

整个知识体系围绕 "AI Agent 是一套围绕目标持续行动的工程系统" 展开，分三大板块七个子域：

- **认知与设计层**（第 1-2 章）回答"是什么、怎么设计"：Agent 的四大能力与判断标准【递进】三种设计模式（ReAct / Reflection / 规划执行）【递进】选型边界（Agent vs Workflow）；
- **工程实现层**（第 3-5 章）回答"靠什么落地、怎么不翻车"：工具设计与 MCP 协议是行动能力的入口【递进】可靠性兜底是生产化前提【并列】记忆与状态管理支撑多步任务；
- **可靠与规模层**（第 6-7 章）回答"好不好、能不能放大"：评估度量让 Agent 可衡量【递进】多 Agent 工程让系统能规模化。

A2 重点难点提示

必掌握核心（重点）

- **Agent 定义与四能力**：规划、工具、执行、反馈；判断标准 = 能否"自己决定下一步"；核心公式 = 目标驱动 + 动态决策 + 外部行动 + 反馈迭代；
- **ReAct 循环**：Thought → Action → Observation → Thought，及其四项工程约束；
- **工具设计六要点**：描述、参数、返回值、粒度、副作用边界、错误恢复——工具决定 Agent 上限；
- **MCP 三角色 + 两层协议 + 三类能力**：Host/Client/Server；数据层（JSON-RPC）与传输层（stdio / Streamable HTTP）；Tools / Resources / Prompts；
- **六层护栏**：任务边界 → 工具边界 → 执行边界 → 状态管理 → 权限控制 → 交付检查；
- **四类记忆**：短期记忆 / Session State / 长期记忆 / 外部知识库（RAG）各司其职；
- **任务完成率四档**：完整完成 / 部分完成 / 正确失败 / 错误失败；
- **DAG 执行器四层**：结构化计划 → 校验排序 → 状态机与 Checkpoint → 局部 Replan（上限 3 次）；
- **三角色分工**：Planner 定义权 / Worker 执行权 / Reviewer 放行权，编排器执行硬约束。

易混易错（难点）

- ⚠️ **Agent vs ChatBot vs RAG 问答 vs Workflow**：区别不在模型更聪明，而在是否"目标驱动 + 动态决策 + 外部行动 + 反馈迭代"；
- ⚠️ **"有分支" ≠ "需要 Agent"**：判断标准是分支规则能否提前写清楚，而不是流程里有没有判断；
- ⚠️ **MCP vs Function Calling**：模型侧调用机制 vs 应用侧接入协议，互补不互斥、互不替代；
- ⚠️ **RAG vs 长期记忆**：向量数据库只是检索层；长期记忆还要写入规则、置信度、更新时间、冲突处理和删除机制；
- ⚠️ **Reflection ≠ 让模型"再想想"**：必须有明确检查标准和外部证据，否则是自我安慰；
- ⚠️ **工具越多 ≠ Agent 越强**：按任务动态收敛工具集；
- ⚠️ **评估不能只看最终答案、安全指标不能被平均**：错误失败（编结果）和正确失败（追问用户）天差地别；

- ⚠️ **Reviewer 不是再做一遍**：是逐条对照执行前定好的验收标准，输出 PASS / REVISE / BLOCKED；
- ⚠️ **消息顺序不按时间戳**：单 Agent 用 seq_no 保序，跨 Agent 看 DAG 依赖和 parent_event_id。

A3 复习建议

1. **按框架自测**：打开思维导图 HTML，从中心主题逐支复述，折叠四级节点先自己回忆再展开核对，卡壳处即薄弱点；
2. **对比记忆**：把 Agent/ChatBot/RAG/Workflow、MCP/Function Calling、RAG/长期记忆、Session State/聊天记录、正确失败/错误失败做成异同对照卡；
3. **场景演练**：拿"定位线上支付接口变慢"这一贯穿全书案例，口述一遍完整设计——先规划什么、ReAct 中间怎么走、Reflection 检查什么、工具怎么设计、失败怎么兜底、评估怎么量化；
4. **易错回扫**：重点重看 A2 标 ⚠️ 的条目，每条配一个自查问题（如："我的分支能提前写清楚吗？"）；
5. **串联复述**：用"认知 → 设计模式 → 工具协议 → 可靠性 → 记忆 → 评估 → 规模化"的逻辑链，把整个体系讲给自己或同伴听一遍——能讲顺，就是真的融会贯通了。

附录 B · 思维导图（树形文本版）

以下树形文本与交互式 HTML 导图内容一致，可直接粘贴进支持文本树的大纲工具。

AI Agent 开发知识体系

- └ 一、Agent 认知
 - └ 本质：围绕目标持续行动的系统
 - └ 普通问答的产物是"答案"【对比】
 - └ Agent 的产物是"完成任务的过程与结果"【对比】
 - └ 四关键词：目标驱动·动态决策·外部行动·反馈迭代
 - └ 四大核心能力【包含】
 - └ 规划：知道任务要怎么拆
 - └ 工具：能连接外部世界
 - └ 执行：一步一步把任务跑完，处理分支
 - └ 反馈：思考→行动→观察结果→再思考
 - └ 判断标准：能否"自己决定下一步"【因果】
 - └ 工具调用路径完全固定 → 只是带工具的 Workflow
 - └ 按任务状态动态决策 → 才是 Agent
 - └ 注意：用了大模型 / Function Calling ≠ Agent
 - └ 与相近形态区分【对比】：ChatBot | RAG 问答 | Workflow
 - └ 不是一个模型，而是一套工程系统【包含】
 - └ 模型·Prompt·工具·执行器
 - └ 状态管理·记忆系统·评估机制
 - └ 权限控制·失败恢复
 - └ 学习地基：结构化输出 + Function Calling
 - └ 输出不可解析 → Agent 直接散架【因果】
 - └ Function Calling 是 Agent 行动能力的入口
- └ 二、设计模式与选型
 - └ 底层共识：Agent 本质是一个循环
 - └ ReAct：边思考、边行动、边观察
 - └ 循环：Thought→Action→Observation→Thought【递进】
 - └ 适用：路径不确定的工具调用任务
 - └ 风险：跑散、忘目标、无效重试【因果】
 - └ 兜底：步数限制·工具权限·状态记录·停止条件
 - └ Reflection：交付前按标准检查
 - └ 适用：易格式错·有验收标准·易过早交付
 - └ 边界：无外部标准时退化为自我安慰【因果】
 - └ 规划执行：先拆任务再推进
 - └ 适用：复杂多步长任务，防跑偏
 - └ 风险：初始计划不一定对【因果】
 - └ 关键：计划是可更新的任务清单
 - └ 三种思路怎么选【对比】
 - └ 路径不确定 → ReAct | 交付前防错 → Reflection
 - └ 复杂多步 → 规划执行
 - └ 真实项目：先规划→中间 ReAct→最后 Reflection
 - └ 选型边界：Agent vs Workflow【对比】
 - └ 判据：能否画成固定流程图 | 分支能否提前写清
 - └ "有分支" ≠ "需要 Agent"
 - └ 能写死就优先 Workflow：更稳、更省、可审计
 - └ 过度 Agent 化：成本高·延迟长·稳定性差·调试难
 - └ 最稳架构：Workflow 控主流程 + 开放节点用 Agent
- └ 三、工具与协议
 - └ 工具=Agent 的行动边界，不是 API 列表
 - └ 工具设计六要点【包含】
 - └ 描述：写清适用场景与不适用场景
 - └ 参数：能枚举就枚举；范围明确；缺参先追问
 - └ 返回值：结构化状态 + 证据 + 下一步提示

- └ 粒度：介于底层 API 与完整业务流程之间
 - └ 副作用：读写分离·高风险二次确认·留审计
 - └ 错误：error_code + recoverable + recommended_action
 - └ 工具集：不是越多越好【对比】→ 按任务动态收敛【因果】
 - └ 五大症状：选错工具·填错参数·不知下一步·重复调用·越界动作
 - └ MCP: Agent 工具调用的新标准
 - └ 定位：标准化连接工具/数据/提示模板的协议
 - └ 三角色：Host · Client · Server【包含】
 - └ 两层协议：数据层 JSON-RPC + 传输层 stdio/HTTP
 - └ 三类能力：Tools 能做事·Resources 能看·Prompts 怎么稳
 - └ 与 Function Calling：模型侧 vs 应用侧【对比】
 - └ 本地 vs 远程：按数据位置与权限边界选【对比】
 - └ 安全：Host 管权限，Server 暴露最小能力
 - └ 一句话：工具设计好了，Agent 才有上限
- └ 四、可靠性兜底
 - └ 根因：自由度带来的必然风险【因果】
 - └ 六类典型翻车【包含】
 - └ 死循环 | 误调用 | 多步中断
 - └ 上下文污染 | 权限越界 | 错误不可恢复
 - └ 死循环兜底【因果】
 - └ 根因：缺停止条件·缺失败记忆·缺完成判断
 - └ 对策：限最大步数·记失败工具·设停止条件
 - └ 误调用兜底：读写分开·二次确认·规则校验
 - └ 模型负责决策，系统负责守门
 - └ 多步中断兜底：外部任务板记录目标/步骤/证据/缺口
 - └ 上下文污染兜底
 - └ 来源：用户猜测·工具噪音·过期上下文·模型假设
 - └ 对策：事实/假设/废弃信息 分层管理
 - └ 权限越界兜底：低风险只读·中风险草稿·高风险须确认
 - └ 六层护栏【递进】
 - └ 任务边界→工具边界→执行边界→状态管理→权限控制→交付检查
 - └ 核心原则：护栏不是限制能力，而是为了能进生产
- └ 五、记忆与状态
 - └ 为什么需要记忆【因果】：多步任务边做边忘 → 不稳定
 - └ 四类记忆【包含】
 - └ 短期记忆：当前对话上下文
 - └ Session State：结构化任务状态
 - └ 长期记忆：跨会话稳定信息
 - └ 外部知识库：常用 RAG 实现
 - └ 短期记忆不是越长越好【对比】→ 应是"当前任务摘要"
 - └ Session State 优于聊天记录【对比】
 - └ 记录：目标·假设·事实·已排除方向·下一步
 - └ 长期记忆必须过筛
 - └ 四问：稳定吗·有用吗·被确认吗·敏感吗
 - └ 像数据库写入，不是聊天记录追加；检索按需
 - └ RAG ≠ 长期记忆【对比】：向量库只是检索层
 - └ 记忆也要会忘：过期·冲突·噪音 → 遗忘机制
 - └ 设计框架【递进】：定类型→定写入规则→按任务检索→用后更新
 - └ 原则：该记的记住，该查的去查，该忘的忘掉
- └ 六、评估与度量
 - └ 为什么难【因果】：答对可能是碰巧 → 看结果也要看轨迹
 - └ 结果层：任务完成率四档【递进】
 - └ 完整完成 | 部分完成 | 正确失败

- └ 错误失败（编结果，最危险）
- └ 过程层【包含】
 - └ 步骤效率：平均步数·无效调用率·重复调用率·关键路径
 - └ 工具调用正确率：选对·参数对·时机对·结果用对
 - └ 错误恢复率：按错误类型分别评估
- └ 安全指标单独统计【对比】：违规率·绕权·泄露·审计完整性
- └ 评估集五类【包含】：正常·缺信息·工具失败·高风险·噪音
- └ 评测方式【并列】：自动化评结构化 | 人工评语义 | judge 只辅助
- └ 线上监控：结合 trace 回放定位跑偏在哪一步
- └ 五大误区【对比】：只看答案·只测正常·只 AI 打分·指标没人看·无归因

└ 七、规模化工程（多 Agent）

- └ Plan-and-Execute 落地成 DAG 执行器【递进】
 - └ 结构化计划：模型输出 nodes + edges
 - └ 校验排序：DFS 三色环检测 + Kahn 拓扑排序
 - └ 并行调度：同层并行，信号量限并发
 - └ 状态机：失败只传播给后继，无关分支照跑
 - └ Checkpoint：按层持久化，重启跳过已完成
 - └ 局部 Replan：只重规划受影响子图，上限 3 次
- └ 三角色分工：拆开定义权/执行权/放行权【包含】
 - └ Planner：任务契约（目标/依赖/工具白名单/验收标准）
 - └ Worker：边界内执行，交付 Artifact+证据+假设+未解决项
 - └ Reviewer：逐条对标准，输出 PASS/REVISE/BLOCKED
 - └ 编排器：确定性代码管状态、权限、返工上限、审计
 - └ 退回路径：执行问题→Worker；拆解问题→Planner；缺权限→人工
 - └ 何时不值得用：任务短可验证/同上下文/标准说不清
- └ 多 Agent 上下文与 Token 治理【包含】
 - └ 独立 Thread：子 Agent 只收任务包，不继承聊天
 - └ Artifact Store：大结果外置，消息只传摘要与引用
 - └ 事件因果：seq_no 保序 + parent_event_id + 幂等键
 - └ 预算原子预留：派发前预留，未用完回收
 - └ 分级降级：60%/80%/95% 三档收束，保护验收额度
 - └ 递归默认关闭：限深度/分支/祖先环/子预算
 - └ 模型路由：按复杂度·风险·可验证性选模型
- └ Browser Agent 读取大型网页【递进】
 - └ 不直塞 DOM：噪声吃 Token，引用易过期
 - └ 按任务选表示：阅读→正文；操作→可访问性树
 - └ 首次只给“页面地图”：landmark/heading/摘要/控件引用
 - └ 语义分块：region→标题树→表格分页→列表游标
 - └ 工具显式分页：truncated + next_cursor + find_in_page
 - └ 增量读取：版本校验 + diff + max_scrolls
 - └ Token 硬约束：单次·累计·滚动次数·输出预留
 - └ 评估：定位质量·任务效果·资源消耗·状态稳定性